

Ideassion Technology Solutions

Name: Durga R

Technology Stack: Springboot

SQL Joins, Aggregate Functions, GROUP BY & HAVING, Advanced SQL Queries

Section A – Theory

1. What is a SQL JOIN? Why do we use JOINS?

A SQL JOIN is used to combine data from two or more tables based on a related column

Why do we use JOINS?

- To retrieve related data from multiple tables.
- To avoid storing duplicate data.
- To create meaningful reports by combining information.

2. Explain the difference between *INNER JOIN, **LEFT JOIN, **RIGHT JOIN, and **FULL OUTER JOIN*.

INNER JOIN	<pre>SELECT * FROM Students INNER JOIN Marks ON Students.StudentID = Marks.StudentID;</pre>
LEFT JOIN	<pre>SELECT * FROM Students LEFT JOIN Marks ON Students.StudentID = Marks.StudentID;</pre>
RIGHT JOIN	<pre>SELECT * FROM Students</pre>

	RIGHT JOIN Marks ON Students.StudentID = Marks.StudentID;
FULL JOIN	SELECT * FROM Students FULL OUTER JOIN Marks ON Students.StudentID = Marks.StudentID;

3. What are Aggregate Functions? Explain the purpose of the following:

Aggregate Functions perform calculations on multiple rows and return a single result.

COUNT()

Returns the number of rows.

```
SELECT COUNT(*) FROM Employees;
```

Total employees = 100

SUM()

Returns the total sum of a numeric column.

```
SELECT SUM(Salary) FROM Employees;
```

Example: Total salary paid = ₹5,00,000

AVG()

Returns the average value.

```
SELECT AVG(Salary) FROM Employees;
```

Example: Average salary = ₹50,000

MIN()

Returns the smallest value.

SELECT MIN(Salary) FROM Employees;

Example: Lowest salary = ₹20,000

MAX()

Returns the largest value.

SELECT MAX(Salary) FROM Employees;

Example: Highest salary = ₹1,00,000

4.What is the purpose of the *GROUP BY* clause

The **GROUP BY** clause is used to group rows that have the same values into summary groups.

It is commonly used with aggregate functions like COUNT(), SUM(), AVG(), MIN(), and MAX().

Example:

SELECT Department, COUNT(*)

FROM Employees

GROUP BY Department

5.Explain the difference between *WHERE* and *HAVING* with an example.

WHERE	HAVING
Filters rows before grouping	Filters groups after grouping
Cannot use aggregate functions directly	Can use aggregate functions
Executed before GROUP BY	Executed after GROUP BY

Example using WHERE

SELECT *

FROM Employees

WHERE Salary > 50000;

Example using HAVING

```
SELECT Department, COUNT(*)  
FROM Employees  
GROUP BY Department  
HAVING COUNT(*) > 5;
```

6.What are Subqueries? Mention one real-world use case.

A Subquery is a query written inside another SQL query.
It helps retrieve data that will be used by the main query.

Example:

```
SELECT Name  
FROM Employees  
WHERE Salary >  
(  
    SELECT AVG(Salary)  
    FROM Employees  
);
```

7. What is the purpose of the *ORDER BY* clause?

The **ORDER BY** clause is used to sort query results in ascending (ASC) or descending (DESC) order.

Example:

```
SELECT *  
FROM Employees  
ORDER BY Salary DESC;
```

Section B – SQL Queries

Write SQL queries to:

- 1. Display all employees along with their department names using an *INNER JOIN*.**

```
SELECT e.Empld,  
       e.Name,  
       d.DepartmentName,  
       e.Salary  
FROM Employee e  
INNER JOIN Department d  
ON e.DepartmentId = d.DepartmentId;
```

- 2. Display all departments, including those that do not have any employees using a *LEFT JOIN*.**

```
SELECT e.Empld,  
       e.Name,  
       d.DepartmentName,  
       e.Salary  
FROM Employee e  
INNER JOIN Department d  
ON e.DepartmentId = d.DepartmentId;
```

- 3. Display the total salary paid for each department.**

```
SELECT e.Empld,  
       e.Name,  
       d.DepartmentName,  
       e.Salary  
FROM Employee e  
INNER JOIN Department d  
ON e.DepartmentId = d.DepartmentId;
```

- 4. Display the average salary of employees in each department.**

```
SELECT e.Empld,
```

e.Name,
d.DepartmentName,
e.Salary

FROM Employee e

INNER JOIN Department d

ON e.DepartmentId = d.DepartmentId;

5. Display the highest salary in each department.

SELECT d.DepartmentName,
 MAX(e.Salary) AS HighestSalary
FROM Employee e
JOIN Department d
ON e.DepartmentId = d.DepartmentId
GROUP BY d.DepartmentName;

6. Display only those departments where the average salary is greater than *55,000*.

SELECT d.DepartmentName,
 MAX(e.Salary) AS HighestSalary
FROM Employee e
JOIN Department d
ON e.DepartmentId = d.DepartmentId
GROUP BY d.DepartmentName;

7. Count the number of employees in each department.

SELECT d.DepartmentName,
 COUNT(e.EmplId) AS EmployeeCount
FROM Department d
LEFT JOIN Employee e
ON d.DepartmentId = e.DepartmentId
GROUP BY d.DepartmentName;

8. Display employees whose salary is greater than the average salary of all employees.

SELECT d.DepartmentName,
 COUNT(e.EmplId) AS EmployeeCount

```

FROM Department d
LEFT JOIN Employee e
ON d.DepartmentId = e.DepartmentId
GROUP BY d.DepartmentName;

```

9. Display the top *3 highest-paid employees*.

```

SELECT d.DepartmentName,
       COUNT(e.EmpId) AS EmployeeCount
FROM Department d
LEFT JOIN Employee e
ON d.DepartmentId = e.DepartmentId
GROUP BY d.DepartmentName;

```

10. Display department names in alphabetical order.

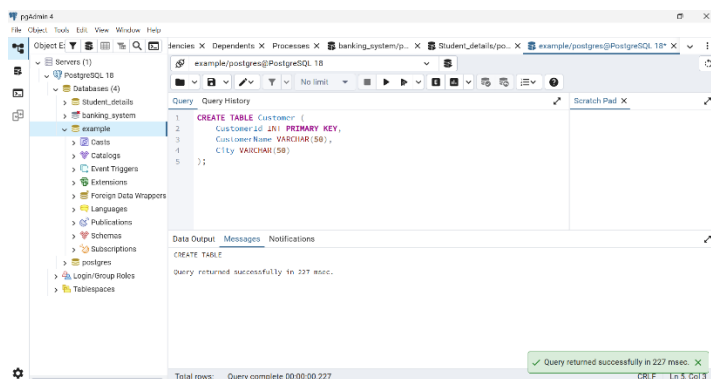
```

SELECT d.DepartmentName,
       COUNT(e.EmpId) AS EmployeeCount
FROM Department d
LEFT JOIN Employee e
ON d.DepartmentId = e.DepartmentId
GROUP BY d.DepartmentName;

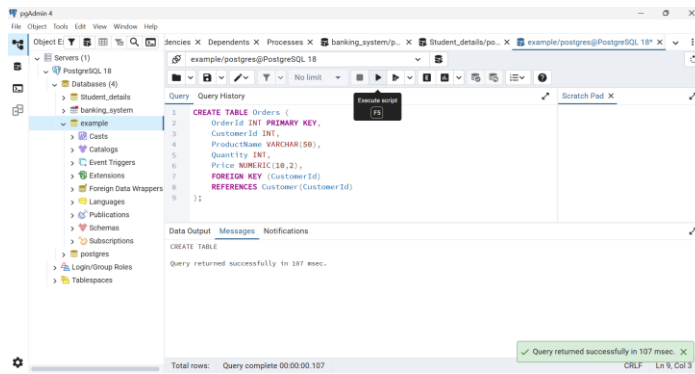
```

Hands-on Practice

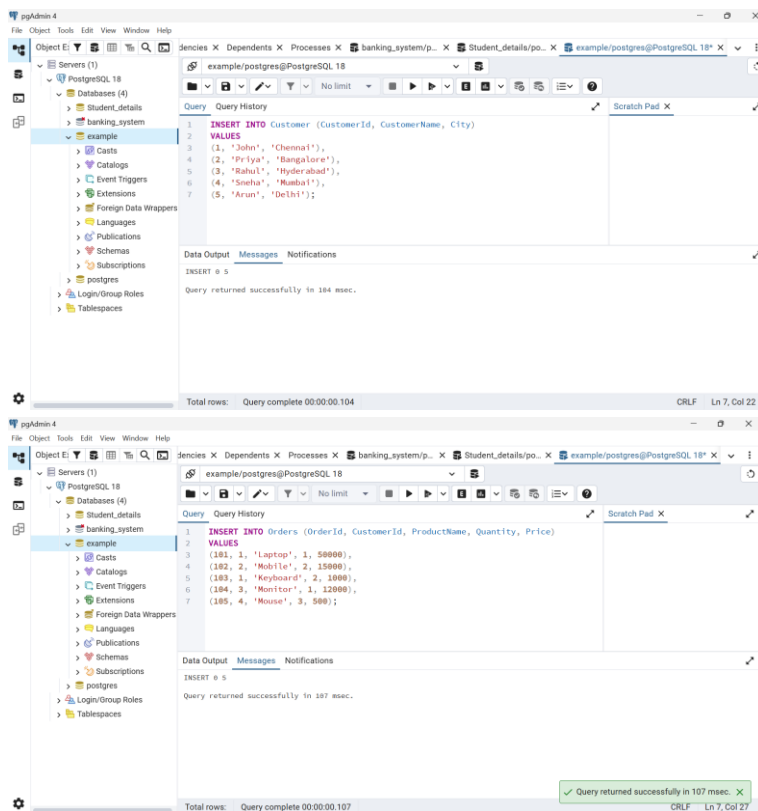
1. Create both tables.



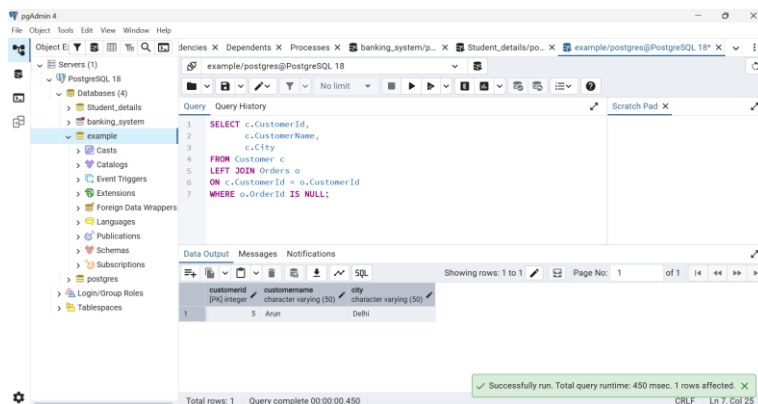
2. Insert at least *5 records* into each table.



3. Display all customer orders using an *INNER JOIN*.



4. Display customers who have not placed any orders.



5. Calculate the total amount spent by each customer.

The screenshot shows the pgAdmin 4 interface with a SQL query executed. The query calculates the total purchase amount for each customer by summing the product of quantity and price for all orders. The results are displayed in a table with columns 'customername' and 'totalamount'.

customername	totalamount
Rahul	12000.00
Shrha	1500.00
John	52000.00
Priya	30000.00

6. Display customers whose total purchase amount is greater than *₹10,000*.

The screenshot shows the pgAdmin 4 interface with a SQL query executed. The query filters the results from the previous query to only include customers whose total purchase amount is greater than 10000. The results are displayed in a table with columns 'customername' and 'totalamount'.

customername	totalamount
Rahul	12000.00
John	52000.00
Priya	30000.00

7. Display the customer who has placed the highest number of orders.

The screenshot shows the pgAdmin 4 interface with a SQL query executed. The query finds the customer with the highest number of orders by counting the number of orders for each customer and sorting the results in descending order. The results are displayed in a table with columns 'customername' and 'ordercount'.

customername	ordercount
John	2

8. Sort customers based on their total purchase amount in descending order.

The screenshot shows the pgAdmin 4 interface with a SQL query executed. The query sorts the results from the first query in descending order of total purchase amount. The results are displayed in a table with columns 'customername' and 'totalamount'.

customername	totalamount
John	52000.00
Priya	30000.00
Rahul	12000.00
Shrha	1500.00

Mini SQL Challenge

1.Find the department with the *highest average salary*.

```
SELECT d.DepartmentName,  
       AVG(e.Salary) AS AverageSalary  
FROM Employee e  
JOIN Department d  
ON e.DepartmentId = d.DepartmentId  
GROUP BY d.DepartmentName  
ORDER BY AverageSalary DESC  
LIMIT 1;
```

2.Find employees earning more than the average salary of their own department.

```
SELECT d.DepartmentName,  
       AVG(e.Salary) AS AverageSalary  
FROM Employee e  
JOIN Department d  
ON e.DepartmentId = d.DepartmentId  
GROUP BY d.DepartmentName  
ORDER BY AverageSalary DESC  
LIMIT 1;
```

3.Display the *top 2 departments* with the highest total salary expenditure.

```
SELECT d.DepartmentName,  
       SUM(e.Salary) AS TotalSalary
```

```
FROM Employee e
JOIN Department d
ON e.DepartmentId = d.DepartmentId
GROUP BY d.DepartmentName
ORDER BY TotalSalary DESC
LIMIT 2;
```

4.Find customers who have placed *more than two orders*.

```
SELECT c.CustomerName,
       COUNT(o.OrderId) AS OrderCount
FROM Customer c
JOIN Orders o
ON c.CustomerId = o.CustomerId
GROUP BY c.CustomerName
HAVING COUNT(o.OrderId) > 2;
```

5.Write a query to display the *second highest salary* from the Employee table.

```
SELECT DISTINCT Salary
FROM Employee
ORDER BY Salary DESC
OFFSET 1 LIMIT 1;
```